Context-aware DAW Assistant: Natural-Language Editing in a DAW

HAI FA26, Assignment 2

Code: github.com/ishaanjagyasi/gemma4-mixer-assistant
Model: huggingface.co/ishaanjagyasi/gemma4-e4b-mixer-assistant
Training data: huggingface.co/datasets/ishaanjagyasi/gemma4-mixer-tool-calls

a. Task

This task and dataset are original to this project (Step D). The model was also fine-tuned specifically for this task.

Each trial presents the participant with two inputs:

- **A music session** loaded in a browser-based DAW modelled on Ableton Live 12 (Figure 1). It contains eight tracks, each with its own volume, pan, send, effect and audio-clip settings.
- **A natural-language request**, for example *"Set send A on track 5 to -15 dB, make the pad a bit louder and add the kick loop to track 3 at bar 14 for 2 bars."*

Purpose of the study. This study is part of *Able to Control*, a broader project on a context-aware voice user interface (VUI) for Ableton Live. Producers spend a substantial share of session time on operational GUI work that competes with creative attention. DAW interfaces are dense and granular, and mouse-and-keyboard interaction ties the musician physically to the workstation.

The study is designed to test whether **voice as an input modality, paired with a context-aware AI that sees the full session state, provides automation benefits over direct mouse-and-keyboard interaction with the GUI**. The benefits examined are accuracy, task completion time and the number of interaction steps.

In this phase, requests are presented as written text standing in for transcribed speech. This isolates the language-understanding step from speech-recognition errors. In Assignment 3, classmates will complete the trials both manually and with the assistant, and their results will be compared with the AI-only results reported here. This tests the predicted ranking of AI only, Human only and Human + AI (Section e).

The participant's goal is to **modify the session so that it matches the request**: every requested change must be made, and no other setting may change. The study has two human conditions:

- **Human only:** participants make the edits manually with the mouse and keyboard.
- **Human + AI:** participants submit the request to the assistant, which applies the edits and lists them. Participants then accept, correct or undo the result.

Each trial contains between 1 and 10 edits, so a participant can complete several trials in a 10-minute session.

How the dataset was made. All trials were generated procedurally by a custom generator (`dataset.js`, `gen.js`; fixed random seeds make the dataset exactly reproducible). Each trial is constructed in four stages:

1. A music session is loaded.
2. A set of edits is sampled from 24 action types, such as lowering the kick's volume or adding a reverb to the snare.
3. The edits are rendered as a single natural-language request, phrased as a producer would express them.
4. The ground truth is recorded: the exact tool calls and the resulting target session state.

A trial is retained only if every edit changes the session state, which guarantees exactly one correct outcome per trial.

Requests are written in one of two registers:

- **Casual:** conversational phrasing, such as *"bring the kick up a touch"* or *"kill the BVs"*. Vague quantifiers map to fixed values: "a touch" = 1 dB, "a bit" = 2 dB, "a lot" = 6 dB.
- **Precise:** explicit numerical values, such as *"set Lead Vox to -7.5 dB"*.

Study Set: The study set comprises 284 trials in two subsets:

Subset	Trials	Description
Standard	200	Typical requests containing 1, 3 or 6 edits; 105 casual and 95 precise
Hard	84	Six request categories absent from the training data, with 14 trials each (listed below)

The six hard categories are:

- **Described tracks:** a track identified by a property rather than by name, e.g. *"the bottom track"*.
- **Relative values:** a value defined relative to another track, e.g. *"5 dB quieter than the kick"*.
- **Unseen slang:** terminology absent from the training data, e.g. *"darken the bass"*.
- **Duplicate track names:** two tracks that share a name, such as two tracks called Snare.
- **Cross-clause references:** an edit that refers to an earlier one, e.g. *"duplicate the vocal and mute the copy"*.
- **Long batches:** requests containing 8 to 10 edits.

Study dataset file: `results/ai/finetuned/results.csv` contains one row per trial, with the following columns:

- **Input:** `instruction`, `initial_state`.
- **Ground truth:** `ground_truth_tool_calls`, `target_state`.
- **Model output:** `model_output_tool_calls`.
- **Evaluation:** `success`, `precision`, `recall`, `f1`, `errors`.

b. The AI assistance and the model

The assistance. The model performs the complete task. Given the request and the current session state, it returns a sequence of tool calls (e.g., `set_volume` or `add_effect`), which the application executes immediately. Following the Guidelines for Human-AI Interaction (Amershi et al., 2019), the user retains control in three ways:

- **Transparency:** the assistant panel lists every change, with its previous and new value.
- **Reversibility:** a single **Undo** reverts all changes made for a request.
- **Direct control:** every control in the interface remains available for manual adjustment.

The model acts only through a fixed set of 24 tools covering mixing, track management, effects, transport and clip editing, so it cannot perform any action the interface does not support.

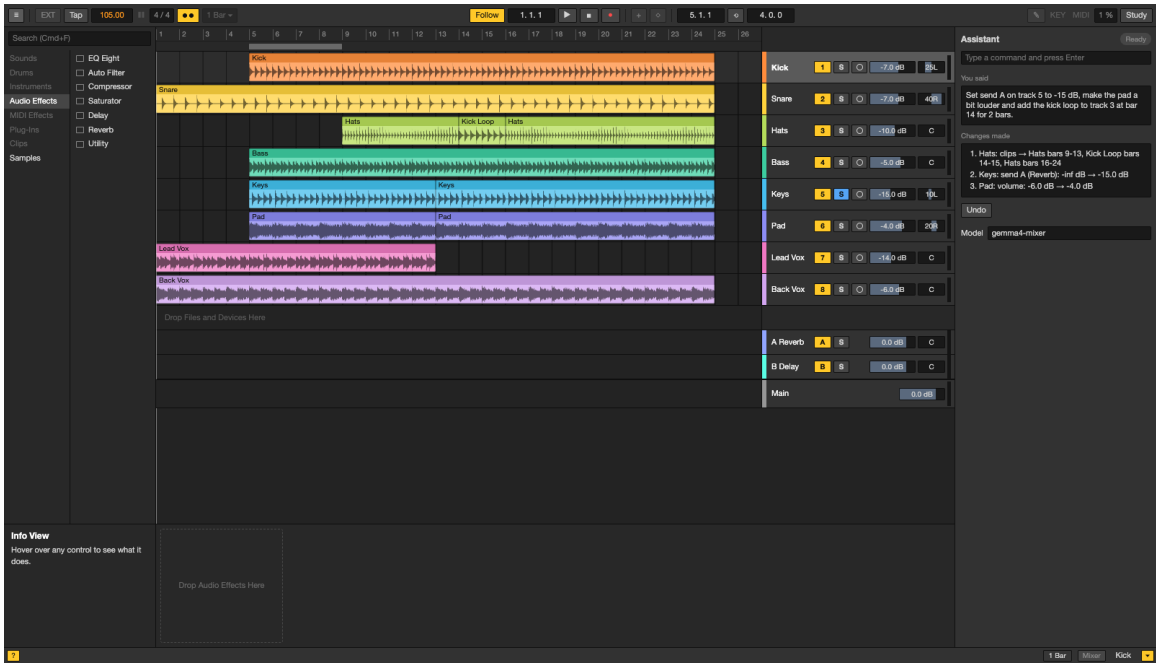


Figure 1. Model output for study trial T077, applied in the application. The assistant panel (right) displays the request and the three changes made by the model, all of which are correct:

- a Kick Loop clip placed on track 3 at bar 14;
- the Keys track's send A set to -15 dB;
- the Pad track's volume raised by 2 dB.

The raw model output for this trial is shown below.

```
Model output (Ollama tool_calls, 5.4 s):
  set_send   {"track": "Keys", "send": "A", "db": -15}
  set_volume {"track": "Pad", "db": 2, "relative": true}
  add_sample {"track": "Hats", "sample": "Kick Loop", "bar": 14, "length_bars": 2}
  -> matches ground truth
```

The model: The assistant uses Gemma 4 E4B, an open-weight model with approximately 4B

effective parameters. It runs locally on a laptop through Ollama and was fine-tuned with LoRA (Hu et al., 2022).

Implementation steps:

1. **Tool and prompt design.** The tool schemas and the prompt (the interpretation rules plus a textual description of the session) are shared by the application, the training data and the evaluation, so the model receives identical input in every setting.
2. **Baseline evaluation.** The untrained model (`ollama pull gemma4:e4b`) completed only 47% of the standard trials correctly.
3. **Training data generation.** 3,000 training and 300 validation examples were generated with separate random seeds. Study trials were excluded, and an automated test verifies this.
4. **Fine-tuning.** The model was fine-tuned on the U-M Great Lakes cluster with Unsloth (LoRA rank 16, one epoch of 750 steps, learning rate 2e-4) on a single A40 GPU for 6.5 hours. The final training loss was 0.0067, and the model answered **98 of 100** held-out validation examples correctly (Figure 2).
5. **Export.** The LoRA adapter was merged into the base model, converted to a 4-bit GGUF file (5.3 GB) with llama.cpp, and registered in Ollama as `gemma4-mixer`.
6. **Deployment.** The model was connected to the application (`server.js`, temperature 0), and the model and training data were published on Hugging Face.

```
[13:03:20] loading unsloth/gemma-4-E4B-it
[13:04:23] 3000 training examples
100%|##########| 750/750 [6:32:27]
{'train_loss': '0.006712', 'epoch': '1'}
[19:37:20] trained in 392.5 min; peak GPU memory 23.7 GB
[19:56:17] validation: 98/100 exactly right

$ ollama list
NAME                  SIZE
gemma4-mixer:latest   5.3 GB
gemma4:e4b            9.5 GB
```

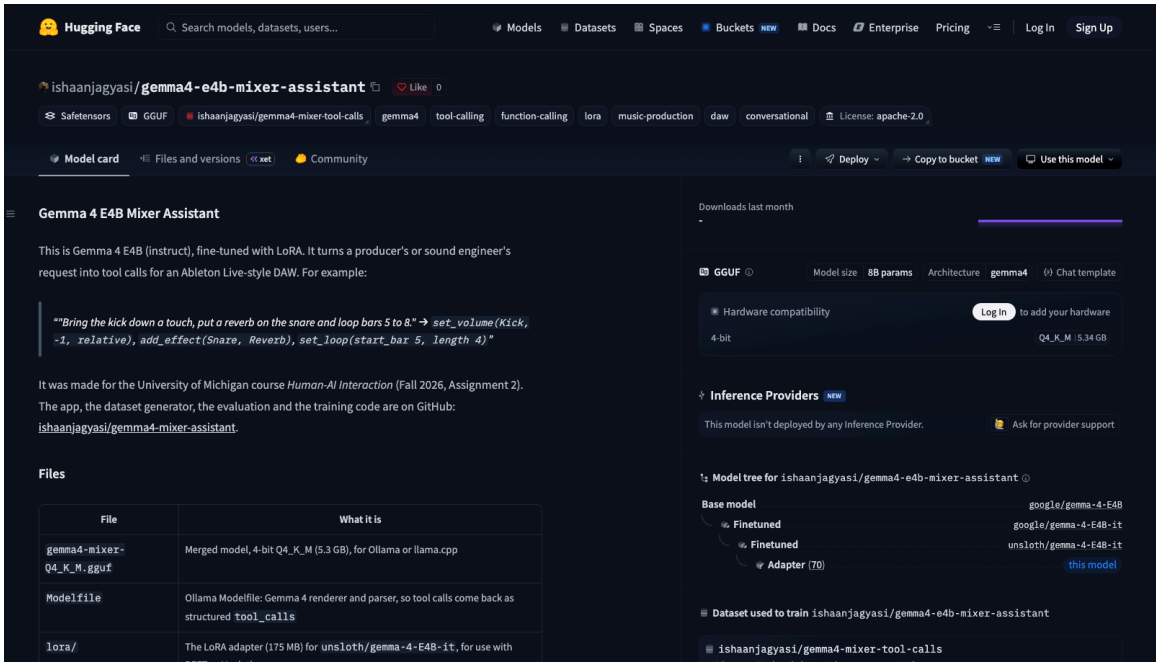


Figure 2. Top: excerpt of the Great Lakes training log (job 64219050) and the models installed in Ollama. Bottom: the published fine-tuned model on Hugging Face.

c. Justification: why users benefit from the assistant but should not delegate to it

Benefits of AI assistance:

- **Efficiency:** a single sentence replaces a long sequence of interface actions (locating the track, opening the device, finding the parameter and entering the value).
- **Precision:** the model executes long, exact requests within seconds and completes 98.9% of precise requests correctly.
- **Domain language:** the model interprets producer terminology, allowing users to stay focused on listening rather than navigating menus.

Reasons not to delegate fully:

- An incorrect edit still yields a valid session, and no warning is raised. The model fails on 12.3% of all trials and on 34.5% of the hard trials.
- It reasons only over a textual description and fails on judgements a person makes at a glance, such as which track is empty, which fader is highest, or what "5 dB quieter than the kick" amounts to.
- Only the user knows what they intended and can judge the result by ear, and the creative work remains theirs.

The model contributes speed and numerical precision; the user contributes visual context and judgement of the result. The change list keeps the cost of verifying the model's output low.

d. Evaluation and error analysis

Setup: All 284 study trials were evaluated with `eval.js`, using the same prompt as the application and a temperature of 0. The model's tool calls are applied to the initial session, and the resulting state is compared with the target session (scoring is implemented in `core.js`).

Metrics:

- **Success (exact match), the primary metric.** A trial succeeds only if the final session equals the target session: every requested change is made and no other setting is altered. The evaluation compares **final states** rather than call sequences, because different sequences of calls can produce the same correct result. This follows the state-based evaluation of tool-using agents in τ-bench (Yao et al., 2024) and the executable evaluation in BFCL (Patil et al., 2025).
- **Precision, recall and F1** over the individual settings that should change. These measures award partial credit, analogous to the exact-match and F1 pair used in SQuAD (Rajpurkar et al., 2016).
- **Tolerances:** 0.25 dB for volume and sends, 1 unit for pan, and 0.25 BPM for tempo.

Trials	n	Untrained success	Fine-tuned success	Fine-tuned F1	Latency (s)
All	284	38.7%	**87.7%**	0.918	7.8
Standard set	200	47.0%	**97.0%**	0.994	5.5
casual / precise	105 / 95	43.8 / 50.5%	95.2 / 98.9%	0.991 / 0.999	
1 / 3 / 6 edits	67 / 67 / 66	62.7 / 46.3 / 31.8%	100 / 98.5 / 92.4%		
Hard set	84	19.0%	**65.5%**	0.736	13.2
described track	14	28.6%	64.3%	0.643	
relative to another track	14	0.0%	14.3%	0.190	
unseen slang	14	21.4%	92.9%	0.964	
same-name tracks	14	14.3%	50.0%	0.661	
refers to earlier edit	14	28.6%	85.7%	0.990	
long batch	14	21.4%	85.7%	0.969	

Error distribution: The fine-tuned model failed 35 trials. Each failure was inspected manually and assigned to its primary cause.

Error pattern	Trials	% of errors	Example (request → model behaviour)	Detectable by a human?
Value defined relative to another track or as a percentage	10	28.6%	"Pad 5 dB quieter than kick" → lowered the pad by 5 dB	Yes: the reference fader is visible on screen
Wrong track among two with the same name	7	20.0%	"Mute the empty keys track" → muted the track containing clips	Yes: clips are visible in the arrangement
Misinterpreted wording	6	17.1%	"Get rid of the reverb" → disabled the effect instead of removing it	Usually, if the change list is read carefully
Track identified by description	5	14.3%	"Set the bottom track to -10 dB" → changed the top track	Yes: the track order is visible
Swapping or mirroring pan positions	4	11.4%	"Swap the pans of snare and lead vocal" → changed only one track	Yes: the change list shows one change instead of two
Reference to a newly created track ("the copy")	2	5.7%	"Duplicate the lead vocal and mute the copy" → referenced a non-existent track	Yes: the application displays an error
Incomplete group edit	1	2.9%	"Clear bars 6 to 11 on everything" → cleared 3 of 8 tracks	Likely: the remaining clips are visible

Counted per incorrect setting, the fine-tuned model's 68 errors are distributed as follows:

- omitted edits: 32%;
- wrong track: 28%;
- wrong value: 18%;
- unrequested changes: 9%;
- invalid tool calls: 6%;
- other: 7%.

The untrained model's errors consisted mainly of omitted edits (43%) and invalid tool calls (27%).

Avoidability by human performers. Most errors are avoidable. Approximately 54% depend on information that is directly visible on screen, and nearly all of the remainder are apparent in the change list. The principal risk is over-reliance (Parasuraman & Riley, 1997): because the model is usually correct, users may stop verifying its output.

e. Prediction

Predicted ranking (exact success): Human + AI > AI only (87.7%) > Human only.

- **AI only > Human only.** Success requires every edit to be exact. Performing 3 to 10 exact edits manually in an unfamiliar DAW leaves considerable room for slips and omitted clauses, whereas the model is nearly error-free on such requests (92–100% on the standard set).
- **Human + AI > AI only.** Participants begin from an output that is correct 88% of the time, and most of its errors are readily visible. Detecting even half of the hard-set errors would place joint performance above that of the model alone.
- **The margin is expected to be small.** Over-reliance, and participants "correcting" outputs that were already right, will both reduce joint performance. On the standard set, Human + AI may only match AI only; the advantage is expected mainly on the hard set.

f. Iterations (Step J)

1. **Model iteration: the base model was insufficient, so a custom model was trained.** The untrained Gemma 4 E4B completed only 47% of the standard trials correctly and produced many invalid tool calls. After fine-tuning, accuracy on the same trials rose to **97%**.
2. **Dataset iteration: the task became too easy for the model, so a hard set was added.** At 97% accuracy, users would have little reason not to delegate. Eighty-four hard trials were added across six categories absent from the training data, without retraining the model. The model scores **65.5%** on this set, and most of its errors are detectable by a human.
3. **Interface iteration: a disambiguation error was corrected.** During testing, with two tracks named "Snare", the request *"delete the one with no audio"* deleted the wrong track. The following changes were made:
    - Tracks that share a name are now labelled "Snare #1" and "Snare #2".
    - An ambiguous track name now produces an explicit error instead of a guess.
    - Each request now includes the current session state.
    - A change list and a one-step Undo were added.

Submitted files

- **Study dataset:** `results/ai/finetuned/results.csv`, together with `data/trials.csv` and the untrained model's results in `results/ai/base/results.csv`.
- **Model and training code:** the application (`app/`, started with `node server.js`), the shared code in `lib/` (`core.js`, `catalog.js`, `prompt.js`, `dataset.js`), `training/` (scripts, Modelfile and logs), `data/train.jsonl`, `data/val.jsonl` and the data generator `scripts/gen.js`. Setup instructions are provided in `README.md`, and the fine-tuned model is available at huggingface.co/ishaanjagyasi/gemma4-e4b-mixer-assistant.
- **Evaluation code:** `scripts/eval.js`, with scoring in `lib/core.js`; the raw model replies and summary tables are in `results/ai/*/predictions.jsonl` and

`summary.md`. Results can be reproduced with `node scripts/eval.js --model gemma4-mixer --label finetuned`, and the tests run with `node --test tests/test.js`.

-

References

1. Amershi, S., et al. (2019). Guidelines for Human-AI Interaction. *CHI 2019*.
2. Hu, E. J., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*.
3. Parasuraman, R., & Riley, V. (1997). Humans and Automation: Use, Misuse, Disuse, Abuse. *Human Factors*, 39(2).
4. Patil, S. G., et al. (2025). The Berkeley Function Calling Leaderboard (BFCL). *ICML 2025*.
5. Rajpurkar, P., et al. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. *EMNLP 2016*.
6. Yao, S., et al. (2024). τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045.